**Second Hand Marketplace**

A full-stack second-hand marketplace application that allows users to create, browse, search, manage, and communicate about advertisements.
The system consists of a Spring Boot backend and a JavaFX desktop frontend connected through RESTful APIs.

**Project Members**

**Amir Hossein Karami**

**Arghavan Mosleh Abady**

**Project Overview**

This project is a second-hand marketplace platform where users can:

- Browse available advertisements without authentication
- Register and login using secure authentication
- Create new advertisements
- Upload multiple images for products
- Edit and manage their own advertisements
- Add advertisements to favorites
- Communicate with sellers
- Rate sellers
- Search advertisements using different filters
- Manage advertisements through an administrator panel

The application separates the frontend and backend layers to provide a scalable and maintainable architecture.

**System Architecture**

The project follows a client-server architecture:

REST API

Frontend ---------- Backend

JavaFX Spring Boot --- Database --- Cloudinary

**Frontend**

- Java 21
- JavaFX
- FXML
- CSS styling

Responsibilities:

- User interface rendering
- User interaction handling
- Sending requests to backend APIs
- Managing user session
- Displaying advertisements and images

**Backend**

- Java 21
- Spring Boot
- Spring Security
- JWT Authentication
- JPA / Hibernate

Responsibilities:

- Business logic
- Authentication and authorization
- Advertisement management
- Image management
- Database communication
- Validation and error handling

**Backend Setup**

**Requirements**

Before running the backend, installed:

- Java JDK 21
- Maven
- Database server

**Configuration**

Update database configuration in:

spring.application.name=secondhand-backend
server.port = 8080
spring.jpa.defer-datasource-initialization = true
spring.datasource.url = jdbc:sqlite:secondhand.db
spring.datasource.driver-class-name = org.sqlite.JDBC
spring.jpa.hibernate.ddl-auto = update
spring.jpa.show-sql = true
spring.jpa.database-platform = org.hibernate.community.dialect.SQLiteDialect

**Cloudinary Configuration**

Images are stored using Cloudinary.

Required properties:

cloudinary.cloud-name=iozk27ac
cloudinary.api-key=413282944439372
cloudinary.api-secret=Z6wUYWZS6bZudwfK7KnFWT9nJTs

**Run Backend**

Run:

mvn spring-boot:run

Backend will start on:

http://localhost:8080

**Frontend Setup**

**Requirements**

- Java JDK 21
- JavaFX SDK

**Run Frontend**

Run the application:

mvn javafx:run

**Database Design**

The database contains several main entities:

**User**

Stores user information:

- Username
- Password
- Phone number
- User role
- Account status

**Product**

Stores advertisements:

- Title
- Description
- Price
- Category
- City
- Status
- Owner

**ProductImage**

Stores advertisement images:

- Image ID
- Cloudinary URL
- Related product

**Other Entities**

- Category
- City
- Favorite
- Conversation
- Message
- Rating

**Authentication and Authorization**

The application uses JWT-based authentication.

Authentication flow:

Login -> Backend verifies credentials -> JWT Token generated -> Frontend stores token

-> Token sent with protected requests

User roles:

- USER
- ADMIN

Access control:

- Users can manage their own advertisements
- Administrators can approve or manage advertisements

**Image Management**

Images are managed using Cloudinary.

Implemented features:

- Upload multiple images
- Store image URLs in database
- Display images in frontend
- Delete images from Cloudinary
- Remove images from advertisements

- Replace or add new images during editing

The path:

User selects image -> Frontend sends Multipart request -> Spring Boot receives image --> Cloudinary Upload -> Frontend displays image

**Testing**

The project was tested using:

**Backend Tests**

Testing includes:

- Authentication endpoints
- Advertisement creation
- Advertisement update
- Advertisement deletion
- Image upload
- Image deletion
- Search functionality

**Frontend Tests**

Testing includes:

- Login/Register flow
- Advertisement browsing
- Creating advertisement
- Editing advertisement
- Adding/removing images
- Logout functionality

**Implemented Features**

**User Features**

- Register

- Login
- Logout
- Browse advertisements
- Search advertisements
- Create advertisement
- Edit advertisement
- Delete advertisement
- Upload product images
- Manage favorites
- Rate sellers
- Send messages

**Admin Features**

- Admin login
- View pending advertisements
- Approve advertisements
- Manage users
- Manage categories and cities

**How backend and frontend connect to each other?**

In the project, the Backend and Frontend are not directly connected; they communicate via REST API and HTTP Requests.

**What establishes the communication in the Backend?**
Controller classes provide the API endpoints.

**What connects in the Frontend?**
In the Frontend, the class ApiClient.java is responsible for sending HTTP requests.

The Frontend and Backend are implemented following a Client-Server Architecture. The communication between the JavaFX Frontend and the Spring Boot Backend is carried out through RESTful APIs and the HTTP protocol. The Frontend sends GET, POST, PUT, and DELETE

requests to the endpoints defined in the Backend's Controllers to retrieve or modify data. Data is transmitted in JSON format, and Multipart Requests are used for sending images.

Here is an example of how a request works:

**Full Path of a Request**

For example, when the user opens the advertisements page:

**1) Frontend**
ProductController calls:
productService.getActiveProducts();

**2) Frontend Service**
ProductService.java builds the request:
GET /api/products

**3) ApiClient**
Completes the full URL:
http://localhost:8080/api/products

**4) Backend Controller**
Receives it with:
@GetMapping

**5) Backend Service**
productService.getActiveProducts()

**6) Repository**
Reads from the database:
productRepository.findByStatus(ProductStatus.ACTIVE)

**7) Response Returns**

| **Database:** | -> | **Backend:** | -> | **Frontend:** |
|---|---|---|---|---|
| Product Data | | JSON | | Jackson converts it to: (ProductDto) and displays it. |

For tests,just simply do:

cd  backend/frontend

mvn test